

JAVA LAB WEEK-8

1. Basic Information:

Title: Interfaces – Employee Payroll using Interface Implementation

Participated Topics:

- Defining an interface
 - Implementing interfaces
 - Accessing implementations through interface references
 - Extending interfaces
-

2. Objective / Aim

To write a Java program using **interfaces** to calculate and display the monthly salary details of different categories of employees in a software company.

The program calculates salary for:

- Regular Employees
- Contract Employees

The program demonstrates:

- Defining an interface
- Extending an interface
- Implementing interfaces
- Accessing class objects using interface references
- Classes, objects, constructors, attributes, and methods

Salary Details

Regular Employee:

- Basic Pay = Rs. 25000
- HRA = Rs. 15000
- T.A = Rs. 5000
- Total Salary = Rs. 45000

Contract Employee:

- Basic Pay = Rs. 12000
 - HRA = Rs. 0
 - T.A = Rs. 3000
 - Total Salary = Rs. 15000
-

3. System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Minimum 1 GB free storage
- Keyboard and Monitor

Software Requirements

- Operating System: Windows / Linux / macOS
 - Java Development Kit: **JDK 17 or JDK 21**
 - IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
 - Command Prompt or Terminal
-

4. Algorithm / Logic Description

1. Start the program.
2. Define an interface named Salary.
3. Declare methods in the Salary interface to calculate and display salary details.
4. Define another interface named EmployeeSalary that **extends** the Salary interface.
5. Create a class RegularEmployee that implements the EmployeeSalary interface.
6. Store the employee ID and regular employee salary values in the class.
7. Create a class ContractEmployee that implements the EmployeeSalary interface.
8. Store the employee ID and contract employee salary values in the class.

9. Read the employee ID from the user.
10. Check whether the employee ID starts with R or C.
11. If the ID starts with R, create a RegularEmployee object.
12. If the ID starts with C, create a ContractEmployee object.
13. Store the object in an EmployeeSalary interface reference.
14. Call the interface method using the interface reference.
15. Calculate the total salary.
16. Display the employee salary details.
17. Stop the program.

OOP Concepts Used

Interface:

Salary defines the salary-related methods.

Interface Inheritance:

EmployeeSalary extends the Salary interface.

Implementation of Interface:

RegularEmployee and ContractEmployee implement EmployeeSalary.

Interface Reference:

An EmployeeSalary reference is used to access objects of RegularEmployee and ContractEmployee.

Constructor:

Constructors initialize the employee ID.

Classes and Objects:

Separate classes and objects are used for regular and contract employees.

5. UML Class Diagram

```
+-----+
|   Salary   |
| <<interface>> |
+-----+
| + calculateSalary() : double|
```

```

| + displaySalary() : void |
+-----+
      ^
      |
      | extends
      |
+-----+
|   EmployeeSalary   |
|   <<interface>>   |
+-----+
| + getEmployeeId() : String |
+-----+
      ^      ^
      |      |
      | implements | implements
      |      |
+-----+ +-----+
| RegularEmployee | | ContractEmployee |
+-----+ +-----+
| - employeeId   | | - employeeId   |
| - basicPay     | | - basicPay     |
| - hra         | | - hra         |
| - ta          | | - ta          |
+-----+ +-----+
| + RegularEmployee | | + ContractEmployee |
| + calculateSalary | | + calculateSalary |
| + displaySalary  | | + displaySalary  |
| + getEmployeeId  | | + getEmployeeId  |
+-----+ +-----+

```

Interface Reference

EmployeeSalary ref;

|
+----> RegularEmployee object
|
+----> ContractEmployee object

ref.displaySalary()

|
v

Actual implementation is executed

6. Source Code

```
import java.util.Scanner;
```

```
// Base interface for salary operations
```

```
interface Salary {
```

```
    double calculateSalary();
```

```
    void displaySalary();
```

```
}
```

```
// Extended interface
```

```
interface EmployeeSalary extends Salary {
```

```
    String getEmployeeId();
```

```
}
```

```
// Regular Employee class
```

```
class RegularEmployee implements EmployeeSalary {
```

```
    private String employeeId;
```

```
    private double basicPay;
```

```
    private double hra;
```

```
    private double ta;
```

```
// Constructor
```

```
RegularEmployee(String employeeId) {
```

```
    this.employeeId = employeeId;
```

```
    this.basicPay = 25000;
```

```
    this.hra = 15000;
```

```
    this.ta = 5000;
```

```
}
```

```
@Override
```

```
public String getEmployeeId() {
```

```
    return employeeId;
```

```
}
```

```
@Override
```

```
public double calculateSalary() {
```

```
    return basicPay + hra + ta;
```

```
}
```

```
@Override  
public void displaySalary() {  
    System.out.println("Salary Details:");  
    System.out.println("Basic Pay: " + (int) basicPay);  
    System.out.println("HRA: " + (int) hra);  
    System.out.println("T.A: " + (int) ta);  
    System.out.println("Total Amount: " + (int) calculateSalary());  
}  
}
```

```
// Contract Employee class
```

```
class ContractEmployee implements EmployeeSalary {
```

```
    private String employeeId;
```

```
    private double basicPay;
```

```
    private double hra;
```

```
    private double ta;
```

```
// Constructor
```

```
ContractEmployee(String employeeId) {
```

```
    this.employeeId = employeeId;
```

```
    this.basicPay = 12000;
```

```
    this.hra = 0;
```

```
    this.ta = 3000;
```

```
}
```

```
@Override
```

```
public String getEmployeeId() {
```

```
        return employeeId;  
    }  
}
```

```
@Override  
public double calculateSalary() {  
    return basicPay + hra + ta;  
}  
}
```

```
@Override  
public void displaySalary() {  
    System.out.println("Salary Details:");  
    System.out.println("Basic Pay: " + (int) basicPay);  
    System.out.println("HRA: " + (int) hra);  
    System.out.println("T.A: " + (int) ta);  
    System.out.println("Total Amount: " + (int) calculateSalary());  
}  
}
```

```
// Main class
```

```
public class EmployeePayroll {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter Employee Id: ");  
        String employeeId = sc.nextLine();  
    }  
}
```

```
// Interface reference
EmployeeSalary employee;

// Create appropriate employee object
if (employeeId.toUpperCase().startsWith("R")) {
    employee = new RegularEmployee(employeeId);
} else if (employeeId.toUpperCase().startsWith("C")) {
    employee = new ContractEmployee(employeeId);
} else {
    System.out.println("Invalid Employee Id");
    sc.close();
    return;
}

// Access implementation through interface reference
employee.displaySalary();

sc.close();
}
}
```

7. Compilation & Execution Commands

Java File Name

EmployeePayroll.java

Compilation Command

javac EmployeePayroll.java

Execution Command

java EmployeePayroll

8. Output Screen

Test Case 1 – Regular Employee

Input:

Enter Employee Id: R101

Output:

Salary Details:

Basic Pay: 25000

HRA: 15000

T.A: 5000

Total Amount: 45000

Test Case 2 – Contract Employee

Input:

Enter Employee Id: C101

Output:

Salary Details:

Basic Pay: 12000

HRA: 0

T.A: 3000

Total Amount: 15000

Test Case 3 – Invalid Employee ID

Input:

Enter Employee Id: X101

Output:

Invalid Employee Id

Observation

For employee ID R101, the program creates a RegularEmployee object and displays:

Basic Pay: 25000

HRA: 15000

T.A: 5000

Total Amount: 45000

For employee ID C101, the program creates a ContractEmployee object and displays:

Basic Pay: 12000

HRA: 0

T.A: 3000

Total Amount: 15000

The objects are accessed using the EmployeeSalary interface reference.

9. Result

The Java program for calculating employee salary using **interfaces** was successfully implemented and executed.

The experiment demonstrated:

- Defining an interface
- Extending an interface
- Implementing interfaces
- Accessing implementations through interface references
- Classes and objects
- Constructors
- Method implementation
- Employee salary calculation